

Analisi Del Bloom Filter

Simone Cappugi
E-mail address

`simone.cappugi@edu.unifi.it`

Codice disponibile:
gitHub

Bloom Filter

Abstract

Il progetto affronta la costruzione di filtri di Bloom per dataset su larga scala. Sebbene l' utilizzo di questi filtri probabilistici è efficiente la loro generazione sequenziale diventa inefficiente con la crescita della dimensione del dataset, poichè vengono calcolate milioni di funzioni di hash. Per migliorare le prestazioni questo studio implementa 5 strategie diverse di parallelizzazione rispetto a una baseline sequenziale usando Python.

1. *Multiprocessing standard utilizzando un pattern MapReduce.*
2. *Multiprocessing con Memoria Condivisa (Shared Memory) per ridurre l'overhead della comunicazione tra processi.*
3. *Joblib utilizzando il patter MapReduce.*
4. *Joblib con Memoria Condivisa (NumPy), un approccio ottimizzato che utilizza il memory mapping per eliminare i costi di serializzazione dei dati durante la distribuzione ai worker.*
5. *Infine è stata implementata una versione con l' utilizzo di `concurrent.futures.ThreadPoolExecutor()`, il quale sfrutta la leggerezza dei thread rispetto alle sue controparti che utilizzano `fork` di vari processi. Questa versione è gestita da `uv ecosystem`.*

Il sistema garantisce la coerenza dei dati attraverso tutte le implementazioni parallele tramite rigorosi unit test.

Risultati chiave (miglior speedup ottenuto): La suite di benchmark dimostra che le implementazioni parallele riducono significativamente il tempo di esecuzione rispetto alla baseline sequenziale. Nello specifico, l'implementazione basata su Joblib con Map Reduce ha ottenuto il miglior speedup (fino ad un max di $\times 2.81$), come evidenziato dai risultati di plot ottenuti. Questo approccio si è rivelato superiore mitigando efficacemente l'overhead di serializzazione tipico dei task di multiprocessing standard, massimizzando l'utilizzo dei core della CPU per i calcoli di hash. L' uso dei thread per questo contesto è stato

tuttavia il migliore, fornendo i risultati migliori in termini di speedup (fino ad un max di $\times 3.4$ e prestazioni.

1. Introduction

Nel contesto della gestione di grandi volumi di dati, come repository di indirizzi email su scala industriale, la verifica efficiente dell'appartenenza a un insieme è un requisito critico. I Filtri di Bloom rappresentano una soluzione standard per questo problema grazie alla loro estrema efficienza spaziale. Tuttavia, la costruzione di questi filtri per dataset contenenti milioni di record (es. 10 milioni di email) pone sfide significative. Sebbene l'interrogazione del filtro sia rapida, la fase di inserimento iniziale ("training") può diventare proibitivamente lenta se eseguita sequenzialmente, limitando la capacità del sistema di aggiornarsi in tempo reale o di gestire flussi di dati massivi. In un ambiente Python standard, l'esecuzione sequenziale può sotto-utilizzare le risorse hardware moderne, lasciando inattiva la maggior parte dei core della CPU. Inoltre, i tentativi di parallelizzazione in Python si scontrano con due ostacoli:

- **Global Interpreter Lock (GIL):** impedisce l' esecuzione simultanea di bytecode python in thread diversi, rendendo inefficace il multithreading come abbiamo affrontato nel primo progetto.
- **Overhead di serializzazione:** è un nuovo elemento introdotto dall' uso del `multiprocessing`, esso richiede la duplicazione dei dati o la serializzazione (pickling) per trasferire dati tra processi. Questa operazione può essere costosa ed eliminare il vantaggio ottenuto con la parallelizzazione.

L' obiettivo di questo progetto è quello di identificare una strategia ottimale per la costruzione dei filtri di bloom in Python eseguiamo quindi vari test sulle performance delle implementazioni cercando di trovare la migliore. Per quanto riguarda il dataset è generato sinteticamente utilizzando la libreria **Faker (localizzazione italiana)** per simulare un ambiente realistico. La classe `EmailManager` offre due modalità di generazione: una **standard**

(nome.cognome@dominio) e una **complessa**, che introduce maggiore entropia aggiungendo suffissi casuali (numeri, anni o ruoli aziendali).

1.0.1 Come si interroga un bloomfilter?

Prima di interrogare il filtro, l'elemento deve essere trasformato esattamente nella stessa forma in cui era quando è stato inserito. Nel codice, questo avviene tramite `em.normalize_email(raw_email)`, che converte tutto in minuscolo e rimuove gli accenti. Questo garantisce che l'algoritmo di hash riceva la stringa corretta. Il dato normalizzato viene passato a k funzioni di hash diverse.

Nel codice, questo viene fatto dal metodo `calculate_hashes`: Il filtro utilizza la libreria **mmh3** (MurmurHash3), che è estremamente veloce e genera un numero intero.

Per simulare k funzioni di hash diverse, il codice passa un "seed" (il parametro `i` nel ciclo `for i in range(k)`) alla funzione **mmh3.hash**. Il risultato dell'hash viene calcolato in modulo m ($\%m$), dove m è la grandezza totale del bit array.

Questo assicura che l'indice generato "cada" sempre all'interno della grandezza dell'array, evitando errori di fuori limite (Out of Bounds). Il risultato è una lista di k indici specifici (ad esempio: [104, 5092, 12, 889]).

Una volta ottenuti i k indici, l'algoritmo controlla l'array di bit in quelle esatte posizioni. La regola (implementata nel metodo `contains` in `BloomFilter.py`) e nel `worker_query_shared` (in `tests\test_queries`) è strettissima: se trova anche solo un bit a 0: l'algoritmo si ferma immediatamente (grazie al `return False` o al `break`) ed è certo al 100% che l'elemento non è mai stato inserito. Questo è il punto di forza del Bloom Filter: non ha mai falsi negativi.

Se trova tutti i k bit a 1: L'algoritmo restituisce `True`. In questo caso, dice che l'elemento è probabilmente presente. Si dice "probabilmente" perché c'è una piccola probabilità statistica (il parametro `PROBABILITY = 0.01`, ovvero 1%) che tutti quei k bit siano stati messi a 1 da altre email diverse inserite in precedenza.

Questo fenomeno è noto come **Falso Positivo**.

1.0.2 Come si crea un bloomfilter?

Prima di inserire qualsiasi dato, il filtro deve sapere quanto deve essere grande. Nel codice, questo avviene solitamente chiamando il metodo di classe `BloomFilter.from_probability(n, p)`. Invece di tirare a indovinare la grandezza dell'array, l'algoritmo usa la matematica per trovare la configurazione perfetta basata su due input:

- **n**(numero stimato di elementi): Quante email prevedi

di inserire (es. 500000)

- **p**(probabilità dei Falsi Positivi desiderata).

Con questi due parametri l'algoritmo calcola due valori:

- **m**(Grandezza dell'array in bit)
- **k** Quante funzioni hash applicare ad ogni singolo elemento per ottimizzare la distribuzione dei bit a 1 e ridurre al minimo le collisioni.

Tuttavia non è l'unico modo per creare un Bloomfilter, si può creare anche attraverso due altre modalità (mai utilizzate in questo progetto).

`Bloomfilter.from_memory(n, m)` (con m il numero massimo di spazio che vogliamo dedicare al bloomfilter).

`Bloomfilter.from_number_of_hashes(n, k)`

(con k il numero di hash che vogliamo impiegare). Una volta eseguiti questi passi il **worker**, per ogni email presente nel dataset di addestramento fa le seguenti operazioni:

- **Normalizzazione:** l'email passa per `em.normalize_email()` (tutto minuscolo, rimozione accenti). Se inserisci "Mario.Rossi@Email.it", il filtro salverà le impronte per "mario.rossi@email.it".
- **Generazione delle impronte:** Il metodo `calculate_hashes(item, m, k)` (in `BloomFilter.py`) prende la stringa normalizzata e la passa alla funzione `MurmurHash3` (`mmh3.hash`). Usando un ciclo da 0 a $k-1$, simula k funzioni hash diverse. Il risultato viene limitato con l'operatore modulo ($\%m$, dove m indica la lunghezza del Bloom Filter) per assicurarsi che i numeri generati siano indici validi per l'array.
- **Accensione dei bit:** Il filtro prende questi k indici (es. [50, 1024, 88]) e va nell'array ad impostare a 1 il bit corrispondente a ciascun indice (`self.bit_array[index] = 1`).

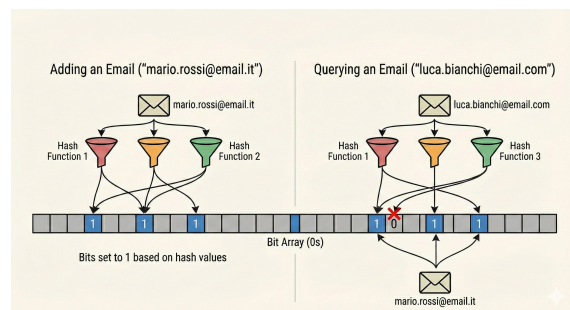


Figure 1. Esempio di Bloom Filter

1.1. Descrizione algoritmo sequenziale

L'algoritmo sequenziale implementato per la costruzione del Filtro di Bloom segue un flusso lineare di elaborazione dati, strutturato principalmente in tre fasi logiche: pre-processing, hashing e inserimento. Dato che questo filtro è pensato per elaborare le email, la fase di normalizzazione è fondamentale. Questa fase è **fondamentale** poichè un malintenzionato, dopo essere stato bannato una volta, (inserito nella lista di email nel Bloom Filter) potrebbe semplicemente creare una nuova email con una maiuscola diversa o con un numero in più e non sarebbe scoperto.

- **Caricamento e Normalizzazione:** Il processo inizia con la lettura del dataset (file CSV contenente indirizzi email). Per ogni email letta, viene invocato l'EmailManager per eseguire la normalizzazione: la stringa viene convertita in minuscolo, vengono rimossi gli spazi e gli accenti per garantire che input visivamente identici producano lo stesso hash. La normalizzazione è fatta solo per far capire a chi vorrà utilizzare i risultati di questo studio che è una parte integrante dell'analisi dei dati. In questo progetto è stata fatta solo un abbozzo, un'idea di quella che è la fase di pre-processing che andrebbe fatta per delle email.
- **Calcolo degli Hash:** Una volta normalizzato l'elemento (email), l'algoritmo riceve in input la probabilità di falsi positivi che si vuole raggiungere (più bassa è la probabilità richiesta e più il filtro sarà costoso in termini di memoria e tempo per il suo calcolo) a questo punto calcola k indici di posizione all'interno dell'array di bit. La classe BloomFilter utilizza la funzione di hash non crittografica MurmurHash3 (mmh3). L'implementazione segue un approccio iterativo: per k volte, viene calcolato l'hash della stringa utilizzando un seed diverso (l'indice del ciclo i) e il risultato viene ridotto modulo m (dimensione del filtro) per ottenere un indice valido. Gli indici k e m sono sempre calcolati in base alla probabilità.

Le formule per calcolare gli indici k (numero di hash necessari per un elemento) ed m (grandezza necessaria del bit array):

$$m = \left\lceil -\frac{n \cdot \ln(p)}{(\ln 2)^2} \right\rceil$$
$$k = \max \left(1, \text{round} \left(\frac{m}{n} \cdot \ln 2 \right) \right)$$

Dove n è il numero atteso di elementi nel filtro e p è la probabilità (*false positive*) richiesta.

1.2. Strategie di Parallelizzazione

L'implementazione del sistema ha previsto l'analisi e lo sviluppo di due scenari di parallelizzazione distinti, volti

a massimizzare l'efficienza della struttura dati in contesti differenti.

- **Fase di Popolamento (Costruzione):** Sebbene i filtri di Bloom siano ottimizzati prevalentemente per operazioni di lettura, l'adozione di strategie efficienti per l'inserimento massivo di dati risulta fondamentale in scenari di *big data*. In questa fase, l'obiettivo è distribuire il carico di hashing e l'aggiornamento del bit-array su più unità di calcolo, riducendo drasticamente il tempo di inizializzazione del filtro.
- **Fase di Interrogazione (Query):** Il secondo scenario riguarda l'esecuzione di lookup simultanei. Parallelizzare le interrogazioni permette di incrementare sensibilmente il *throughput* del sistema, rendendolo capace di gestire un volume elevato di richieste concorrenti, aspetto cruciale per applicazioni come il filtraggio di pacchetti di rete o la verifica di database di email su larga scala.

1.2.1 Approccio adottato per l'inserimento in bloom-filter

Per quanto riguarda la parallelizzazione a primo colpo sembra intuitiva l'applicazione del pattern di *MapReduce*.

- **Fase di Map:** le operazioni di **normalizzazione** e di **calcolo degli hash** sono completamente indipendenti dalle altre email nel dataset. Il dataset può essere diviso in *chunk* arbitrari e distribuiti su più unità di calcolo (si chiamano *worker* in questo pattern) nella fase di sviluppo iniziale saranno *processi*, poi verranno utilizzati *thread*.
- **Fase di Reduce:** sezione che si occupa di prendere i risultati calcolati in locale dai vari **mapper worker** e unirli in un unico bitarray. Viene utilizzato un **bitwise OR** per ottenere un risultato globale finale.

Quindi grazie all'utilizzo di numpy è stata utilizzata la vettorizzazione per eliminare i colli di bottiglia nella fase di aggregazione e per gestire efficientemente la memoria, lasciando iterativa solo la parte di calcolo dell'hash (che è intrinsecamente sequenziale per singolo elemento).

L'istruzione: `np.bitwise_or(array, arr_view, out=final_array)`, questa singola riga esegue l'OR logico su milioni di bit in parallelo a livello di CPU (SIMD), risultando ordini di grandezza più veloce di un ciclo Python manuale.

Questo approccio in fase di sviluppo è stato sostituito con l'utilizzo di *bitarray* che hanno permesso di risparmiare molta memoria in fase di esecuzione. L'operazione utilizzata per aggregare i risultati è: `|=`. In generale viene sempre seguita questa idea.

Ci saranno tuttavia delle implementazioni che utilizzano versioni chiamate: **"Shared"**. In queste implementazioni viene utilizzata una memoria condivisa nella quale è consentita la presenza di race condition. I worker scrivono il valore 1 direttamente nella memoria condivisa senza utilizzare alcun meccanismo di **lock**. In questo caso il lock non è necessario poiché ogni operazione consiste semplicemente nella scrittura del valore "1", e non esiste il rischio di inconsistenze dovute a scritture concorrenti con valori differenti. Occorre tuttavia prestare particolare attenzione alle differenze tra le versioni che utilizzano processi e quelle che utilizzano thread. Nel caso dei **processi**, è **necessario richiedere al sistema operativo porzioni di memoria specificamente dedicate alla condivisione tra processi**, poiché, per impostazione predefinita, essi non condividono lo spazio di memoria e non possono comunicare direttamente tra loro.

Nel caso dei **thread**, invece, è **stato sufficiente creare un oggetto `bytearray` e utilizzarlo come memoria condivisa. Non sono state necessarie ulteriori operazioni grazie alla natura stessa dei thread**, che condividono lo stesso spazio di indirizzamento del processo.

È stato scelto di utilizzare un `bytearray` condiviso invece di un `bitarray` per evitare possibili problemi dovuti a race condition. Infatti, la CPU opera tipicamente su unità di memoria di dimensione pari ad almeno un byte: la lettura e la modifica di singoli bit richiederebbero operazioni di lettura-modifica-scrittura sul byte contenente il bit stesso. In presenza di più thread concorrenti, questo potrebbe generare conflitti e risultati inconsistenti. Utilizzando un `bytearray`, invece, ogni thread modifica direttamente un byte indipendente, riducendo il rischio di tali interferenze.

1.2.2 Interrogazione bloom filter

Mi sono affidato a due metodi principali. Il primo è il metodo `run_query_benchmark_sequential` (in `tests/test_queries.py`, il quale definisce la baseline prestazionale del sistema. Questa funzione funge da punto di riferimento per valutare l'efficienza computazionale del Filtro di Bloom su un singolo core, escludendo qualsiasi overhead derivante dalla sincronizzazione tra processi. Il flusso esecutivo prevede un'iterazione sequenziale attraverso le liste degli elementi **presenti e assenti**

Per ogni record il sistema provvede alla normalizzazione della stringa tramite il modulo `EmailManager` prima di interrogare il filtro. La precisione della misurazione è garantita dall'uso di `perf_counter()`, che cattura i tempi di esecuzione dell'intero ciclo di scansione, permettendo così di calcolare il throughput finale espresso in query al secondo. Ovviamente il risultato atteso è che la lista dei *presenti* (elementi già inseriti nel bloom filter) abbia un

throughput più basso rispetto a *assenti*. Questo perché non sarà necessario effettuare tutti i confronti hash richiesti, risparmiando tempo.

Il secondo metodo `run_query_benchmark_parallel` è stato progettato per massimizzare il rendimento sfruttando l'architettura multi-core della CPU, con un focus specifico sulla scalabilità in ambienti *No-GIL*. La procedura inizia con una fase di analisi del sistema, in cui viene verificato lo stato del *Global Interpreter Lock* per segnalare all'utente eventuali colli di bottiglia dovuti alla contesa del lock nell'interprete standard. Una volta superata questa fase, il carico di lavoro viene ripartito tramite una strategia di *chunking*, suddividendo i dataset in segmenti proporzionali al numero di thread/processi rilevati in quell'esempio. Ad ogni thread/processo viene assegnato un singolo chunk. L'esecuzione parallela viene gestita da un pool di thread che invocano simultaneamente le routine di query su ogni porzione di dati. Infine, il metodo aggrega i risultati parziali man mano che i thread completano i compiti assegnati, fornendo una metrica complessiva della capacità di elaborazione massiva del sistema sotto sforzo concorrente.

1.3. Architettura Software: Orchestrator e Worker

Questo capitolo vuole spiegare come ho applicato il pattern Map Reduce spiegato poco prima.

Per abbattere i tempi di costruzione del Bloom Filter, specialmente con dataset dell'ordine di milioni di elementi, il sistema adotta un'architettura concorrente basata sul pattern *Master-Worker*. Questa logica è stata astratta e separata in due moduli distinti: `orchestrator.py`, che funge da coordinatore globale (Master), e `worker.py`, che definisce le unità di calcolo atomiche (Worker).

1.3.1 L'Orchestratore "`orchestrator.py`"

La classe `BloomOrchestrator` funge da centro di controllo per la parallelizzazione. Il suo scopo principale è gestire il ciclo di vita del Bloom Filter, partizionare il carico di lavoro e coordinare l'esecuzione dei vari processi o thread. Le sue responsabilità principali includono:

- **Inizializzazione del Filtro:** All'avvio, l'orchestratore calcola i parametri ottimali (m bit e k funzioni hash) istanziando il Bloom Filter globale in base alla probabilità di falso positivo desiderata.
- **Partizionamento dei Dati (Data Split):** Tramite il metodo `split_data`, il dataset originale viene diviso in *chunk* (sotto-liste) di dimensioni eque. Il numero di chunk è calcolato in base al numero di worker disponibili moltiplicato per un fattore di granularità.
- **Gestione dei Paradigmi Concorrenti:** L'orchestratore espone dei metodi basati su:

- multiprocessing.Pool
- joblib.Parallel
- concurrent.futures.ThreadPoolExecutor

- **Fase di Reduce (Merge):** Nei paradigmi standard (senza memoria condivisa), l'orchestratore attende che tutti i worker restituiscano i propri array di bit locali e li unisce applicando un operatore logico OR bit a bit ($|=$).

1.3.2 Il Ruolo del Worker (`worker.py`)

Il file `worker.py` contiene le funzioni pure che vengono iniettate nei processi o thread paralleli. Ogni worker riceve in input esclusivamente il proprio *chunk* di dati, la dimensione del filtro (m) e il numero di hash (k). Il suo compito prevede:

1. **Normalizzazione:** Per ogni stringa grezza ricevuta, il worker invoca l'`EmailManager` per standardizzarla (lowercase, rimozione accenti).
2. **Hashing:** Calcola i k indici univoci sfruttando la funzione `MurmurHash3`.
3. **Aggiornamento dei Bit:** La modalità di aggiornamento cambia in base alla strategia:
 - *Approccio Isolato (Map-Reduce):* Il worker alloca un `bitarray` locale inizializzato a zero, accende i bit e restituisce la struttura al Master.
 - *Approccio a Memoria Condivisa (Shared Memory):* Il worker non alloca memoria locale. Tramite riferimenti a oggetti `multiprocessing.Array` o agganciandosi a `shared_memory.SharedMemory` di NumPy, il worker scrive direttamente (e simultaneamente) le variazioni di stato sulla memoria centrale. Essendo il Bloom Filter una struttura a scritture puramente additive, questo approccio risulta *lock-free*.

2. Setup per gli esperimenti

Per quanto riguarda le informazioni della macchina su cui verrà eseguito il codice:

Processore: Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz

RAM: 8,00 GB (7,78 GB utilizzabile).

GPU: Intel(R) Iris(R) Plus Graphics.

Sistema Operativo: Windows 11.

Interprete:

- Python 3.10/3.11 (Standard): Utilizzato per le versioni sequenziali e basate su multiprocessing. In questo ambiente, il parallelismo reale è ottenuto tramite processi separati per ovviare alle limitazioni del GIL.

- Python 3.14.2 free-threading build: Utilizzato specificamente per lo script `main_no_gil.py`, `\test\test_no_gil.py`. Questa versione dell'interprete permette l'esecuzione di thread multipli in parallelo su core diversi senza l'ostacolo del GIL (PEP 703).

Strumento di misura: `time.time()`

Metrica di test: Misurato il tempo di esecuzione della normalizzazione e calcolo dei vari hash.

Procedura di test:

- Ripetizione dell'esperimento per N volte.
- Calcolo della media aritmetica dei tempi per ridurre la varianza dovuta al jitter del sistema operativo.
- Incrementato il numero di worker (vengono avviati più processi) dopo aver eseguito N volte lo stesso esperimento.

3. Validazione della Correttezza

Per quanto riguarda la verifica della correttezza ho eseguito due **test di unità** fondamentali.

3.1. Validazione Algoritmica

Il primo livello di verifica, implementato in `\unittest\verify_correctness.py`, accerta che le proprietà del Filtro di Bloom siano rispettate. I test controllano due invarianti:

- **Assenza di Falsi Negativi:** Viene verificato che ogni elemento inserito nel set di training restituisca `True` quando interrogato. Un fallimento qui indica un errore critico nella funzione di hash o nella scrittura dei bit.
- **Tolleranza sui Falsi Positivi (FPR):** Viene misurato empiricamente il tasso di falsi positivi su un set di elementi disgiunti. Il test asserisce che l'FPR misurato non superi la soglia teorica definita ($P = 0.01$) oltre una tolleranza di sicurezza (+1.5%), validando il dimensionamento dei parametri m e k .

3.2. Coerenza dell'Implementazione Parallela

Il secondo livello di verifica che viene gestito da `\unittest\verify_consistency.py`, garantisce che l'introduzione del parallelismo non alteri il determinismo del risultato. Viene generato il *bit array* sequenziale e confrontato con l'output delle diverse strategie

parallele (Multiprocessing, Joblib, Shared Memory...). Il test esegue un confronto bit a bit (*bitwise equality*) dell'intero array:

$$\text{BitArray}_{seq} \equiv \text{BitArray}_{parallel}$$

L'uguaglianza rigorosa conferma l'assenza di *race conditions* durante la fase di scrittura concorrente nella memoria condivisa o durante il merge (fase di reduce) dei risultati parziali.

4. Confronto tra tecniche di parallelizzazione

In questo lavoro ho raccolto risultati da due versioni principali:

- **Processi**
- **Thread**

Per semplicità sono stati divisi in due file differenti `test.py` per quanto riguarda i processi e `test_no_gil.py` per quanto riguarda la gestione del parallelismo con i thread.

4.1. Parallelismo con processi

In `tests.py` sono stati affrontati e studiati 4 particolari casi:

1. **Multiprocessing Standard (Map-Reduce):** Implementato tramite `multiprocessing.Pool`. Il dataset viene diviso in chunk e inviato ai worker. Ogni worker genera un oggetto `bitarray` locale e lo restituisce al processo Master, che si occupa di fonderli tramite un'operazione di OR logico. Questa variante, seppur concettualmente semplice, soffre dell'overhead di comunicazione inter-processo (IPC) dovuto alla serializzazione (pickling) e deserializzazione delle stringhe e degli array.
2. **Multiprocessing con Memoria Condivisa (`multiprocessing.Array`):** Per mitigare i costi di IPC della versione standard, questa implementazione pre-allocava un array di byte condiviso a livello di sistema operativo. I worker ricevono solo i chunk di dati e un riferimento alla memoria condivisa, effettuando scritture concorrenti *lock-free*.
3. **Joblib Standard:** Una variante del pattern Map-Reduce che sfrutta la libreria `joblib.Parallel`. Joblib ottimizza la distribuzione dei task (spesso tramite backend *loky*), ma richiede comunque il ritorno e l'aggregazione di `bitarray` separati.
4. **Joblib con Memoria Condivisa (NumPy Shared Memory):** Rappresenta l'approccio più avanzato

tra quelli basati sui processi. Utilizza il modulo `multiprocessing.shared_memory` introdotto in Python 3.8 per allocare un blocco di memoria raw, che viene poi mappato (memory-mapped) in un `numpy.ndarray` di interi a 8-bit. I worker Joblib scrivono direttamente in questo array condiviso in tempo reale, abbattendo quasi a zero il costo di aggregazione finale.

Ho testato questi esempi con vari dataset:

```
DATASETS_FILES = ["dataset_10k.csv", "dataset_100k.csv", "dataset_500k.csv", "dataset_1.5m.csv", "dataset_3m.csv", "dataset_5m.csv", "dataset_10m.csv"]
```

4.2. Analisi dei Risultati: Parallelismo basato su Processi

I grafici riportati in Fig. 2 riguardano la creazione dei Bloom Filter, essi mostrano l'andamento del tempo di esecuzione e dello *Speedup* al variare del numero di worker (da 1 a 16) su dataset di dimensioni crescenti (da 10.000 a 10 milioni di email). L'analisi di queste metriche rivela dinamiche fondamentali riguardanti il bilanciamento tra carico computazionale e costi di comunicazione.

4.2.1 L'Overhead (Dataset 10k e 100k)

Osservando i risultati per i dataset più piccoli (10k e 100k), emerge un dato controintuitivo: **l'esecuzione parallela è nettamente più lenta della baseline sequenziale**. Nei grafici dello speedup, tutte le curve si posizionano al di sotto della soglia $1\times$. Questo fenomeno è dovuto al costo di *Inter-Process Communication* (IPC) e allo *startup time* dei processi. Il tempo necessario al sistema operativo per allocare nuovi processi, serializzare i dati e trasferirli supera abbondantemente il tempo effettivo di calcolo degli hash (che nel sequenziale a 10k impiega appena 0.03s). Inoltre, all'aumentare dei worker, la latenza di coordinamento peggiora, spiegando la pendenza positiva (peggiorativa) della curva dei tempi.

4.2.2 Il superamento (500k - 10m)

A partire dal dataset di 500k elementi, il carico computazionale (milioni di chiamate a `MurmurHash3`) diventa sufficientemente grande da "nascondere" l'overhead IPC. Nei dataset massivi (1.5m, 3m, 5m, 10m) l'esecuzione parallela mostra finalmente il suo valore, offrendo tempi di completamento inferiori rispetto al sequenziale.

4.2.3 Limite Hardware e Context Switching

In tutti i grafici in cui si registra un vantaggio prestazionale, si nota che la curva dello speedup cresce seguendo vaga-

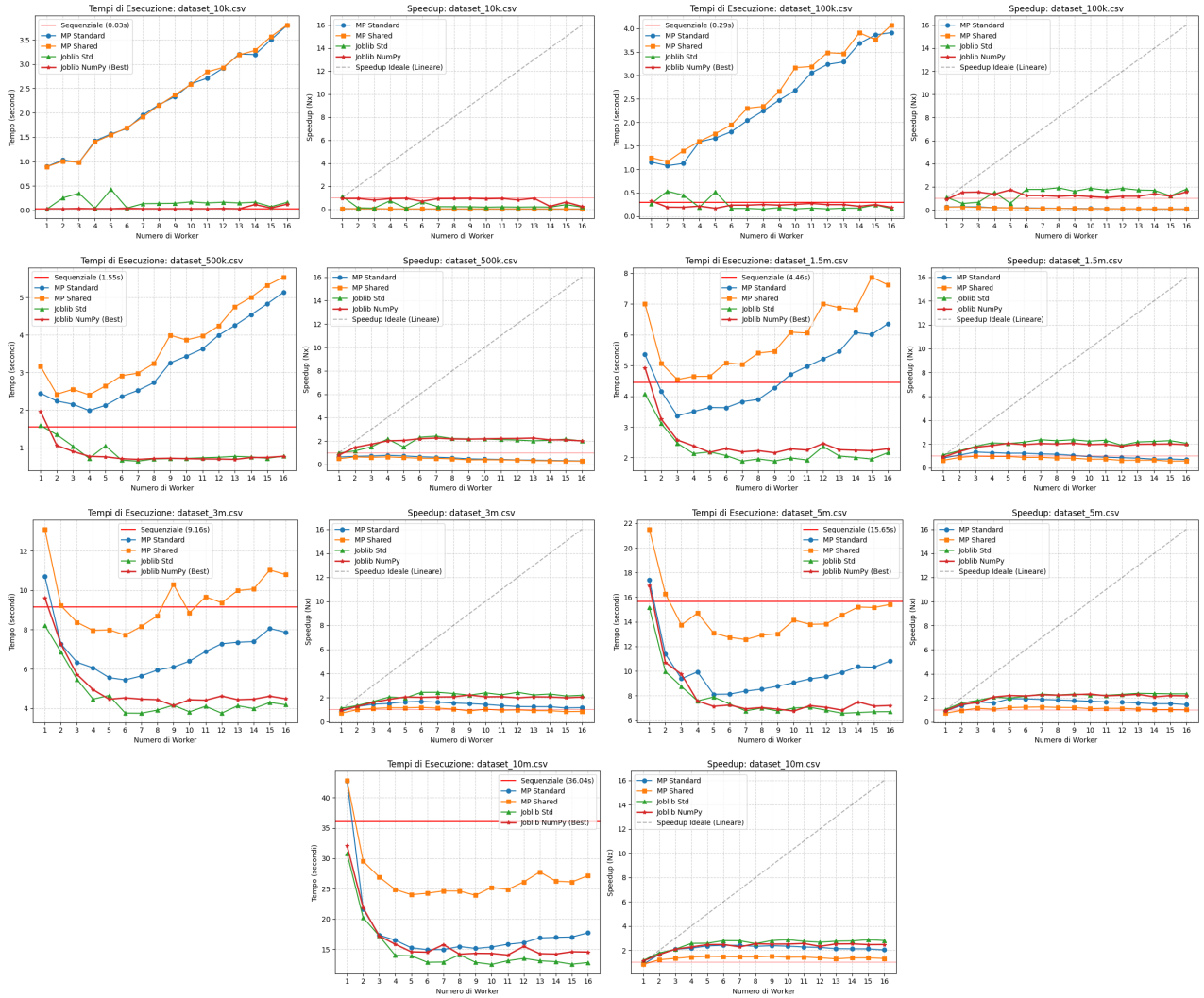


Figure 2. Risultati dei test con dataset di dimensioni crescenti: 10k, 100k, 500k, 1.5M, 3M e 5M email.

mente la linea ideale solo nei primissimi step, per poi appiattirsi (plateau) o addirittura degradare superata la soglia dei 4-8 worker. Questo comportamento è direttamente riconducibile alle specifiche della macchina di test, equipaggiata con un processore **Intel Core i7-1065G7** (4 core fisici, 8 thread logici). Avere 16 worker attivi su un processore quad-core non fornisce ulteriore parallelismo reale, ma innesca un fenomeno di *Context Switching thrashing*: il sistema operativo perde cicli di clock preziosi per alternare l'esecuzione dei troppi processi in coda sui medesimi core fisici. Il picco di efficienza si assesta coerentemente tra i 4 e gli 8 worker.

4.2.4 Confronto tra le Strategie Implementate

L'aspetto più interessante dell'analisi è la marcata divergenza tra le quattro strategie implementate:

- **Joblib (Std e NumPy):** Risultano costantemente i performer migliori. Joblib utilizza sotto il cofano il backend *loky*, che è altamente ottimizzato per il riutilizzo dei processi (evitando di respawarli) e per la gestione intelligente della memoria condivisa. L'integrazione tra Joblib e NumPy Shared Memory ha registrato i picchi massimi di *speedup* (sfiorando il 3× sui dataset più grandi). Negli esperimenti successivi ho forzato il **non riutilizzo** dei processi nelle varie iterazioni di prova dell' algoritmo mediante il comando: `get_reusable_executor().shutdown(wait=True)`. In questo modo i Joblib e Multiprocessing sono più simili come funzionamento. Immagini del risultato: Fig:3. Il risultato è molto diverso, joblib viene forzato a non riutare gli stessi processi ma a generarne sempre di nuovi, molto simile a come funziona il `with in`

multiprocessing. Joblib risulta comunque la scelta migliore tra le due metodologie.

- **MP Standard (Map-Reduce):** Mostra prestazioni discrete, ma inizia a degradare molto prima rispetto a Joblib all'aumentare dei worker.
- **MP Shared (multiprocessing.Array):** Paradossalmente, l'approccio nativo di Python per la memoria condivisa si è rivelato il peggiore in quasi tutti gli scenari, posizionandosi spesso sotto le performance del sequenziale. Nonostante l'assenza di lock espliciti (`lock=False`). L'accesso ripetuto e granulare (scrittura di milioni di singoli bit in un ciclo) genera un overhead applicativo immenso rispetto all'accesso diretto a livello C garantito, invece, dagli array NumPy in `SharedMemory`.

4.2.5 Shared vs MapReduce (Std)

Per confrontare nel dettaglio l'efficienza delle due architetture parallele, è stata condotta un'analisi granulare dei tempi di esecuzione avvalendosi della libreria `line_profiler`. Applicando il decoratore `@profile` ai metodi critici dell'orchestratore (annotazioni mantenute commentate nel codice sorgente). Il file coinvolto da lanciare con `kernprof -l profiling\prof_macro.py` fornisce i risultati del profiling coinvolto nella Table 1. Il comando per visualizzare i risultati è `python -m line_profiler -rmt prof_macro.py.lprof`

I risultati confermano la netta superiorità dell'approccio *Standard* (basato sul paradigma Map-Reduce). Questa predominanza deriva dall'efficienza intrinseca nell'utilizzo di strutture dati native: i worker della versione *Standard* allocano blocchi di memoria locale di tipo `bitarray` (un'operazione di istanziazione quasi istantanea), mentre l'approccio *Shared* richiede l'allocazione di un blocco di memoria condivisa inter-processo (IPC) strutturato in `Byte`, il cui tempo di setup iniziale risulta fisiologicamente maggiore.

Tuttavia, l'analisi ha rivelato che la vera criticità del modello a memoria condivisa risiede nella fase finale di "Reduce" (ovvero la conversione da `bytearray` a `biarray`). L'operazione di conversione dell'enorme buffer IPC (basato su array di `Byte`) in un vero e proprio `bitarray` (requisito fondamentale per il Bloom Filter finale) introduce un *overhead* prestazionale imprevisto e altamente penalizzante. Questa singola operazione arriva a saturare circa il 26.6% del tempo totale di esecuzione dell'orchestratore, vanificando di fatto i benefici teorici derivanti dall'eliminazione del passaggio di parametri tra i processi. Solo i **risultati più significativi** sono espressi nella Table 1 (il test è svolto con un **dataset di 500k** elementi e l'uso di **4 worker**). Il test

è stato svolto su `multiprocessing` in versione MapReduce e `Shared Memory`. Mediante l'utilizzo di una IA generativa mi sono fatto aiutare sull'ottimizzazione di questa parte di codice.

L'ottimizzazione prevede di:

- **creare** una vista numpy sulla memoria condivisa.
- **impacchettare** l'array con `np.packbits()` funzione interamente scritta in C molto efficiente.
- **travasare** la memoria grezza direttamente nel `bitarray` con `bitarray.frombytes()`.
- **slicing**, mediante l'uso di `final_bloom[:self.bloom.m]` per rispettare la grandezza vera del filtro di bloom e non avere padding ottenuto da `np.packbits()`.

I risultati ottenuti mostrano un miglioramento notevole, quasi fino a pareggiare le performance ottenute con la versione Map reduce (std). Risultati in Table 2

4.3. Parallelismo con thread

Per esplorare il parallelismo a memoria condivisa senza l'overhead dei processi IPC, è stata adottata la versione sperimentale **Python 3.14.2 (free-threaded build)**.

Questa particolare distribuzione permette la rimozione del GIL, consentendo a più thread di **accedere simultaneamente alla stessa istanza del Bloom Filter in RAM senza conflitti di interpretazione**.

Per la gestione di questo ambiente d'avanguardia è stato utilizzato **uv**. L'utilizzo di `uv` è stato fondamentale per:

- Isolare l'interprete *free-threaded* dal resto del sistema.
- Garantire l'installazione di librerie compatibili (come `numpy` e `bitarray`) compilate specificamente per girare senza il blocco del GIL.

In `test_no_gil.py` sono state analizzate tre ulteriori casistiche basate su `ThreadPoolExecutor`:

1. **Threaded Merge (Bitarray):** Applica il pattern Map-Reduce all'interno dello stesso processo. Ogni thread alloca un oggetto `bitarray` locale per processare il proprio chunk. Al termine, l'orchestratore esegue la *merge* dei risultati tramite l'operatore OR logico.
2. **Threaded Merge (ByteArray):** Variante della precedente che utilizza `bytearray` locali (array di interi a 8-bit) durante la fase di calcolo. Questa scelta introduce un *overhead* significativo: il consumo di RAM aumenta di 8 volte e la conversione finale da `bytearray` a `bitarray` risulta particolarmente onerosa.

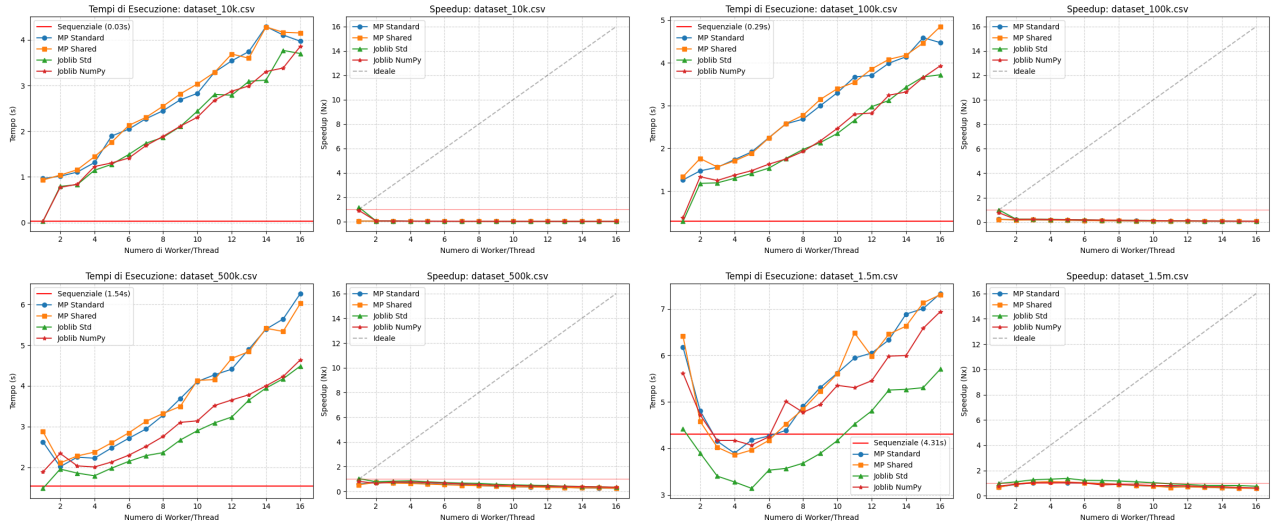


Figure 3. Risultati dei test con dataset di dimensioni crescenti: 10k, 100k, 500k, 1.5M email con `get_reusable_executor().shutdown(wait=True)` abilitato.

Table 1. Confronto dei tempi di esecuzione: Analisi delle fasi dell'Orchestratore

Fase dell'Orchestratore	Tempo (μ s)	% Tempo	Operazione Chiave (Codice)
Versione Map-Reduce (Standard)			
Creazione Array Locale	158.4	0.0%	<code>global_bitarray = bitarray(m)</code>
Setup Pool Multiprocessing	31 172.0	2.6%	<code>with Pool(...) as pool:</code>
Attesa Worker (Fase di Reduce)	1 167 334.4	96.8%	<code>for ba in results:</code>
Unione Finale (Fase di reduce)	1 576.3	0.1%	<code>global_bitarray = ba</code>
Versione Memoria Condivisa (Shared)			
Allocazione Memoria IPC	4 724.2	0.29%	<code>Array('b', m, lock=False)</code>
Setup Pool Multiprocessing	30 815.0	1.86%	<code>with Pool(...) as pool:</code>
Distribuzione e Attesa Calcolo	1 176 689.3	71.13%	<code>pool.map(worker.process..., args)</code>
Conversione Lenta in Bitarray	442 068.5	26.73%	<code>final_bloom.extend(shared_bit_array)</code>

Table 2. Confronto dei tempi di esecuzione: Versione Ottimizzata con NumPy

Fase dell'Orchestratore	Tempo (μ s)	% Tempo	Operazione Chiave (Codice)
Versione Memoria Condivisa (Shared - Ottimizzata NumPy)			
Allocazione Memoria IPC	6 379.6	0.5%	<code>Array('b', m, lock=False)</code>
Distribuzione e Attesa Calcolo	1 135 811.9	96.5%	<code>pool.map(worker.process..., args)</code>
Impacchettamento ("C")	1 066.5	0.1%	<code>np.packbits(arr.view, bitorder='big')</code>
Travaso Memoria Zero-Copy	183.7	0.0%	<code>final_bloom.frombytes(...)</code>

3. Threaded Shared (NumPy): Rappresenta l'approccio più innovativo reso possibile da Python 3.14. Viene istanziato un unico `numpy.ndarray` condiviso nativamente tra tutti i thread nello stesso spazio di indirizzamento. Grazie all'assenza del GIL, i thread effettuano scritture concorrenti sull'array

globale (non devo chiedere al SO memoria condivisa). Questa architettura abbate drasticamente i tempi di latenza, eliminando sia l'allocazione di memoria IPC (necessaria nel multiprocessing), sia la fase di aggregazione finale, poiché il Bloom Filter è già integro al termine dell'ultimo thread.

Ho testato questi esempi con vari dataset:

```
DATASETS_FILES = ["dataset_10k.csv", "dataset_100k.csv", "dataset_500k.csv", "dataset_1.5m.csv", "dataset_3m.csv", "dataset_5m.csv"]
```

4.4. Analisi dei risultati: Parallelismo basato su thread

I grafici riportati in Fig: 4 mostrano l'andamento del tempo di esecuzione e dello *Speedup* al variare del numero di worker (da 1 a 16) su dataset di dimensioni crescenti (da 10.000 a 5 milioni di email). L'analisi di queste metriche rivela dinamiche fondamentali riguardanti il bilanciamento tra carico computazionale e costi di comunicazione. Rispetto al caso di utilizzo di processi con i thread notiamo subito miglioramenti al tempo di esecuzione.

Infatti già subito al primo caso si nota che le curve relative ai casi paralleli stanno al di sotto della baseline sequenziale. Come ben si può notare abbiamo un notevole vantaggio in termini di tempo. I casi che utilizzano thread tendono ad essere più veloci rispetto ai processi per questo tipo specifico di task.

Probabilmente questa differenza è dovuta all'assenza del **pickling** e dalla velocità di spawn dei thread.

Mostro nella Table 3 i risultati ottenuti dalla profilazione. Questa profilazione è stata svolta manualmente. Ho fatto un commit su GitHub dove mostro questo passaggio per poi averlo tolto e riportato il codice alla versione iniziale.

La profilazione mostrata nella tabella è stata ottenuta con **4 thread** e un dataset di email pari a 500k.

Table 3. Confronto Prestazionale: Threaded Merge vs. Shared Thread (500k elementi)

Fase dell'Esecuzione	Merge (ms)	Shared (ms)
Setup & Split	4.53	4.37
Allocazione Memoria	—	0.05
Calcolo Thread (Core)	447.06	632.34
Fase Finale (Merge/Conv.)	0.99	103.42
TEMPO TOTALE	452.58	740.18

I dati riportati evidenziano una netta discrepanza nel **Core Work** (Calcolo % più veloce della *Shared*). Questo conferma l'impatto del *False Sharing* sulla coerenza delle cache L1/L2 quando più thread operano in scrittura su un unico array NumPy.

Inoltre, la versione *Shared* è ulteriormente penalizzata dal costo di conversione finale (103.42 ms), rendendo l'approccio *Standard Merge* la soluzione ottimale in ambiente Python 3.14 *free-threaded*. Seguendo la stessa logica usata per ottimizzare la conversione finale nel caso dei processi ho ottenuto questi risultati:

Table 4. Confronto Prestazionale Finale: Strategie Multithreading Ottimizzate (500k elementi)

Fase dell'Esecuzione	Merge (ms)	Shared Opt. (ms)
Setup & Split	3.60	3.60
Calcolo Thread (Core)	447.06	629.83
Fase Finale (Merge/Conv.)	0.99	1.08
TEMPO TOTALE	451.65	634.57

Il risultato in Table 4 (assunto Allocazione di Memoria ~ 0) dimostra che il vantaggio derivante dall'eliminazione della fase di aggregazione (*Reduce*) non è sufficiente a compensare il degrado prestazionale causato dal *False Sharing* e dalla minore efficienza di scrittura di *numpy* rispetto alle strutture *bitarray* native. Si conclude pertanto che l'architettura a memoria locale con *merge* finale rappresenta la **soluzione ottimale** per il parallelismo.

Nella Table 4 e 3:

- **Setup & Split:** Rappresenta il tempo necessario per preparare i dati iniziali. In questa fase il dataset completo viene suddiviso in piccoli pezzi (chunk) per essere distribuito tra i vari processori (thread).
- **Calcolo Thread (Core):** È il "cuore" del lavoro (Core Work). Indica il tempo speso dai thread per calcolare le funzioni di hash e identificare quali posizioni attivare nel filtro. In un sistema parallelo questa è la parte che dovrebbe beneficiare più l'aumento di thread! Guarda immagini 5,6!
- **Fase Finale (Merge/Conv.):** Misura il tempo necessario per consolidare il lavoro svolto dai singoli thread in un unico risultato finale.

Confrontiamo ora nella Table 5 i valori della versione *bitarray* vs *bytearray*. Entrambi usano 4 thread e un dataset da 500k email.

Nella fase di Merge nella versione *bytearray* viene utilizzato *NumPy Bitwise OR*, che richiede più tempo del velocissimo *or* usato in *bitarray*. I dati presentati nella Table 5 utilizzano la versione ottimizzata per la conversione finale.

Table 5. Confronto Prestazionale: Threaded Merge (Bitarray vs. ByteArray)

Fase dell'Esecuzione	Bitarray (ms)	ByteArray (ms)
Setup & Split	4.16	3.96
Calcolo Thread (Core)	462.81	472.65
Fase di Merge	0.92	41.46
Conversione Finale	—	0.98
TEMPO TOTALE	467.90	519.09

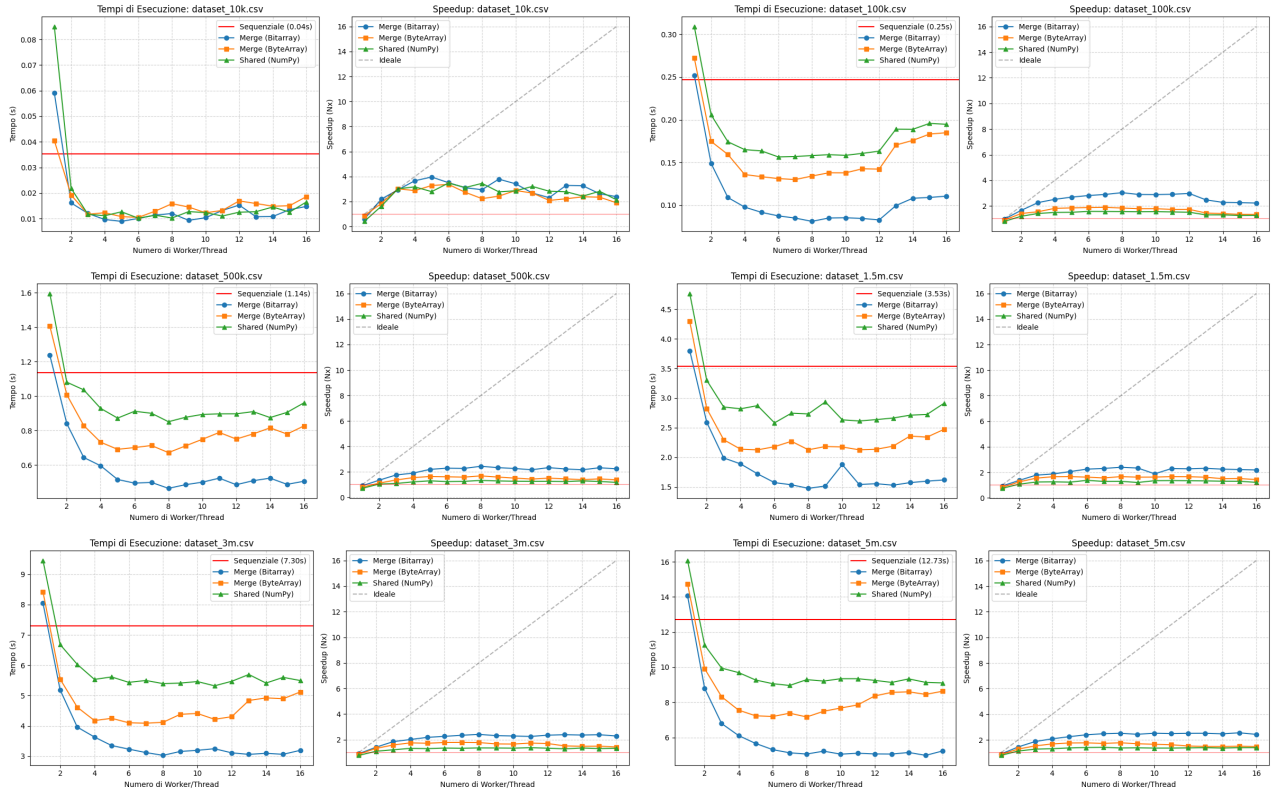


Figure 4. Risultati dei test con dataset di dimensioni crescenti: 10k, 100k, 500k, 1.5M, 3M e 5M email.

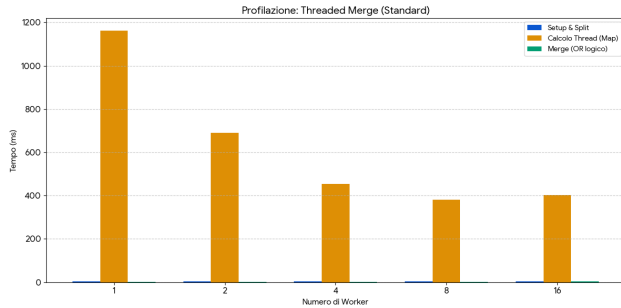


Figure 5. Profilazione con più thread caso standard

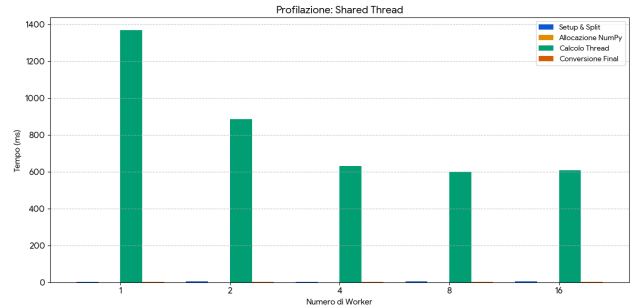


Figure 6. Profilazione con più thread caso Shared

5. Interrogazione Bloom Filter

In questa sezione testeremo l' utilizzo di un bloom filter in parallelo.

Dato che l' utilizzo più frequente del bloom filter sarà proprio l' interrogazione ho predisposto un analisi per questo scenario.

5.1. Struttura dell' esperimento

In questa fase del progetto, l'obiettivo è misurare le prestazioni del sistema in fase di ricerca. A differenza della fase di popolamento. È stato implementato un modulo che

confronta tre approcci distinti: Sequenziale, Multithreading (No-GIL) e Multiprocessing con Shared Memory.

- **Baseline Sequenziale:** Fornisce il termine di paragone eseguendo le query su un singolo core. Rappresenta il limite inferiore delle prestazioni.
- **Multithreading** (Python 3.14 Free-threaded): Sfrutta la nuova build di Python per eseguire query parallele sullo stesso oggetto BloomFilter. Questa tecnica è teoricamente la più efficiente per le query, poiché i thread condividono il bitarray senza alcuna necessità di

comunicazione inter-processo o mappatura di memoria aggiuntiva.

- **Multiprocessing con Shared Memory:** Per i sistemi Python standard (dove il GIL è attivo), è stata implementata una versione che utilizza `multiprocessing.shared_memory`. Il *bitarray* viene copiato in un segmento di memoria condivisa del sistema operativo e mappato come un `numpy.ndarray`. I worker calcolano gli indici di hash in modo indipendente e interrogano direttamente la memoria condivisa. In questo caso ho utilizzato: `concurrent.futures.ProcessPoolExecutor` per rimanere molto allineato al codice fatto per i Thread.

5.2. Metriche di Valutazione

Il parametro principale di valutazione è il **Throughput**, misurato in query al secondo (q/s).

- **Query su elementi presenti:** Questi elementi sono tutti già stati configurati nel Bloom Filter, richiedono il calcolo di **tutti** i k hash e il controllo di tutti i bit corrispondenti (*costo massimo*).
- **Query su elementi assenti:** Questi elementi invece non sono sicuramente nel Bloom Filter. Beneficiano dell'ottimizzazione del Bloom Filter, che **interrompe** il controllo non appena trova il primo bit a 0.

5.3. Analisi Qualitativa e Verifica della Correttezza

Oltre alle prestazioni, il modulo integra un sistema di validazione per garantire l'integrità dei dati:

- **Verifica Falsi Negativi:** Conferma che il tasso di errore sia pari a zero. Un falso negativo indicherebbe una corruzione della memoria condivisa o un bug nell'algoritmo di hash.

5.4. Analisi dei risultati

I risultati della Table 6 mostrano l'andamento del throughput al variare della dimensione del set di addestramento e di test.

Table 6. Benchmark: 500k addestramento / 100k test

Worker	Thread (q/s)		Process (q/s)	
	Presenti	Assenti	Presenti	Assenti
Base Seq.	368.984	444.571	—	
1	326.026	394.485	102.261	112.947
2	621.340	679.145	114.790	123.535
4	777.218	1.163.040	108.311	105.923
8	846.552	1.282.814	69.835	69.417
16	808.296	1.234.522	37.854	40.445

Visivamente abbiamo il seguente Speedup:

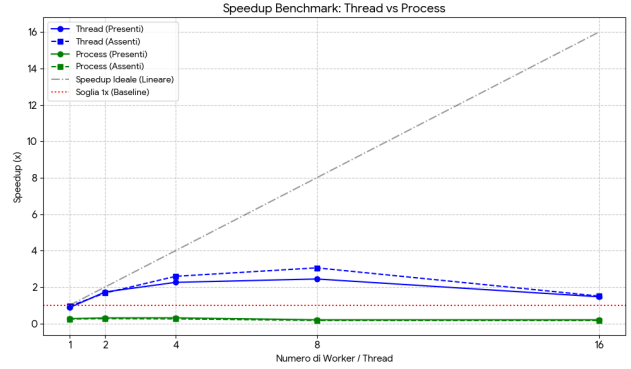


Figure 7. Speedup "email presenti" e "email assenti"

Visivamente abbiamo il seguente throughput:

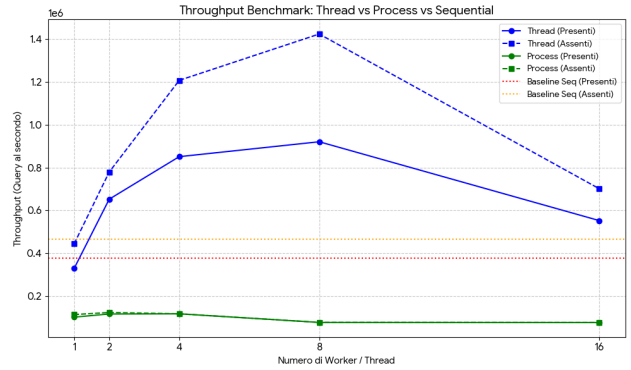


Figure 8. Throughput "email presenti" e "email assenti"

Risultato nella Table 7 è ottenuto con un BloomFilter addestrato con 1m email e testato su 200k email.

Table 7. Benchmark: 1M addestramento / 200k test

Worker	Thread (q/s)		Process (q/s)	
	Presenti	Assenti	Presenti	Assenti
Base Seq.	350.581	415.456	—	
1	322.185	388.469	147.224	168.111
2	564.033	699.782	184.768	208.897
4	835.551	1.132.684	198.041	193.106
8	841.216	1.264.575	126.216	127.474
16	810.238	1.196.917	64.326	69.452

Visivamente abbiamo il seguente Speedup:

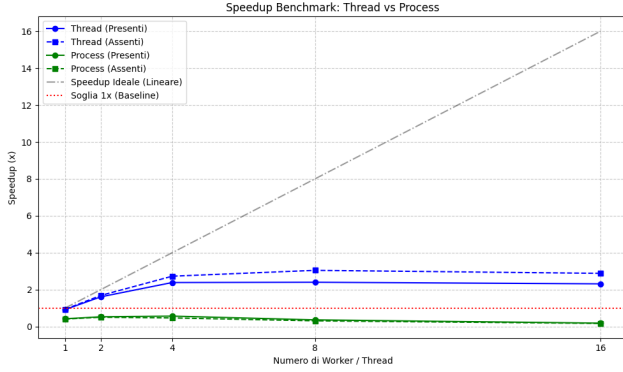


Figure 9. Speedup "email presenti" e "email assenti"

Visivamente abbiamo il seguente throughput:

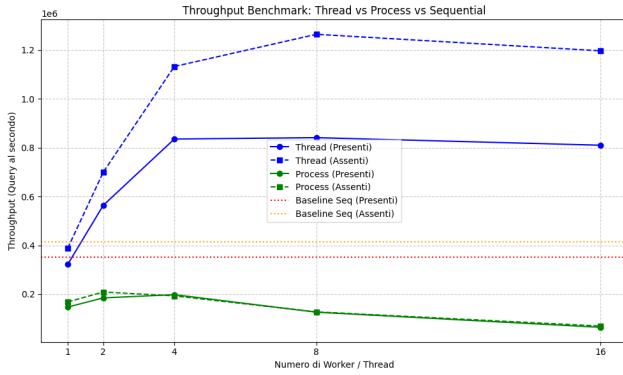


Figure 10. Throughput "email presenti" e "email assenti"

Risultato nella Table 8 è ottenuto con un BloomFilter addestrato con 3m email e testato su 200k email.

Table 8. Analisi del Throughput: 3M Training / 200k Test

Worker	Thread (q/s)		Process (q/s)	
	Presenti	Assenti	Presenti	Assenti
Base Seq.	320.381	396.700	—	
1	305.261	370.552	140.825	169.218
2	499.295	713.465	175.358	205.397
4	830.688	1.165.878	180.928	189.947
8	816.323	1.256.585	124.980	127.752
16	749.334	1.216.061	67.085	69.029

Ho ingrandito ulteriormente il test set (portandolo ad 1mln di elementi) per verificare la scalabilità dei processi. In questo scenario i Processi iniziano a mostrare un vantaggio rispetto alla baseline sequenziale. Table 9

Table 9. Analisi del Throughput su Test Set di 1mln elementi

Worker	Thread (q/s)		Process (q/s)	
	Presenti	Assenti	Presenti	Assenti
Base Seq.	322.417	411.757	—	
1	314.993	363.471	212.547	277.258
2	542.775	692.724	336.156	441.430
4	804.838	1.038.092	443.779	528.081
8	833.558	1.191.917	369.907	422.673
16	774.987	1.111.429	250.190	255.743

Visivamente abbiamo il seguente Speedup:

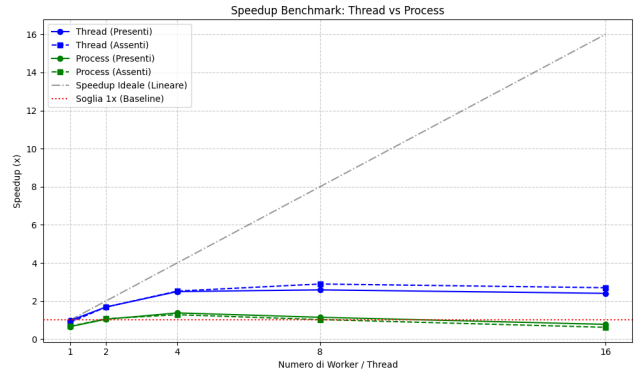


Figure 11. Speedup "email presenti" e "email assenti"

Visivamente abbiamo il seguente throughput:

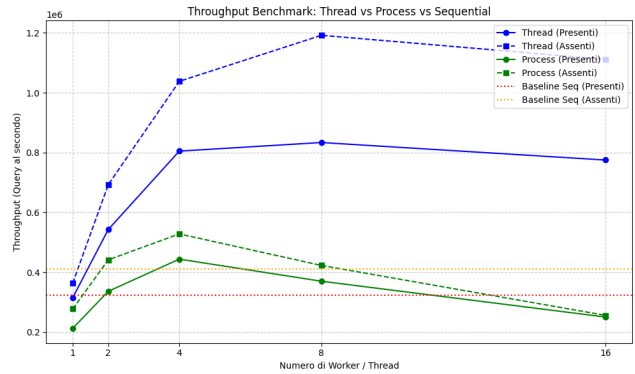


Figure 12. Throughput "email presenti" e "email assenti"

Con 1 milione di query complessive, il tempo di calcolo ammortizza l'overhead di creazione dei processi, riuscendo quasi a raggiungere la versione sequenziale.

5.5. Conclusioni sui Benchmark

Dall'analisi complessiva dei test condotti variando sia la dimensione del filtro (addestramento) che il carico di interrogazioni (test set), emergono tre considerazioni fondamentali sull'uso del Bloom Filter in Python:

- **Il trionfo del Free-Threading (No-GIL):** L'utilizzo dei thread in Python 3.14 con GIL disabilitato si è dimostrato l'approccio largamente più efficiente in ogni scenario testato. Le query di lettura, non richiedendo lock strutturali sul *bitarray*, scalano in modo quasi lineare fino a saturare la banda passante della memoria della CPU (solitamente tra gli 8 e i 16 worker), garantendo speedup fino a 3× rispetto all'esecuzione sequenziale pura.
- **L'inefficienza cronica dei Processi (Shared Memory):** L'approccio basato sul modulo `multiprocessing.sharedmemory` ha registrato performance nettamente inferiori in tutti i contesti. Il costo architetturale di instradamento dei dati (IPC) e la sincronizzazione di accesso a un array NumPy esterno da parte di processi separati vanifica i benefici del parallelismo, rendendo i processi spesso più lenti della semplice esecuzione sequenziale.
- **Ammortamento dell'overhead su larghi volumi:** Come evidenziato nell'ultimo test (Tabella 9 con test set da 1 milione di elementi), l'unico scenario in cui i processi multiprocesso riescono a superare la baseline sequenziale è quando il volume delle query è sufficientemente massiccio. Solo in questo caso limite il tempo di calcolo puramente computazionale riesce a compensare l'ingente overhead iniziale di creazione dei processi e clonazione della memoria condivisa.

5.6. Dimostrazione delle conclusioni

Per comprendere l'inefficienza riscontrata nell'architettura a memoria condivisa, ho condotto una profilazione isolando i tempi di esecuzione delle singole fasi (Tabella 10) il codice è disponibile in `prof_queries.py`. Il test è stato eseguito interrogando 200.000 email distribuite su 4 worker, avvalendosi di un Bloom Filter addestrato su 1 milione di elementi.

Table 10. Micro-profilazione delle Query: Thread vs. Processi (1M Train / 200k Test su 4 Worker)

Fase dell'Esecuzione	Thread (ms)	Processi (ms)
Creazione SHM & Copia Dati	—	312.27
Setup & Lancio Pool	0.89	0.99
Connessione Worker a SHM	—	0.26
Esecuzione Reale (Core)	217.78	217.17
Overhead (Pickling e Sync)	2.22	901.62
TEMPO TOTALE	220.88	1432.05

Spiegazione dei punti di profiling della Tabella 10.
Per i thread:

- **Setup & Lancio Pool:** È il tempo impiegato da Python per inizializzare il `ThreadPoolExecutor` e "smistare"

i blocchi di dati (chunk) ai vari thread.

- **Esecuzione Reale (Core):** È il tempo netto trascorso dal thread più lento a fare il lavoro utile. Consiste nel prendere la stringa, calcolarne le funzioni di hash (tramite `mmh3`) e controllare se i bit corrispondenti nel Bloom Filter sono impostati a 1 o 0.
- **Overhead:** È la differenza tra il tempo totale del programma e il tempo di "esecuzione reale".

Per i processi:

- **Creazione SHM & Copia:** È il tempo che il sistema operativo impiega per creare un'area di "Shared Memory" dedicata e il tempo impiegato da Python per copiarvi all'interno l'intero array di bit del Bloom Filter.
- **Lancio Process Pool:** Il tempo speso per creare (effettuare il fork o lo spawn) di nuovi processi fisici a livello di sistema operativo.
- **Connessione Worker a SHM (Media per worker):** Una volta che i processi worker sono nati, devono "agganciarsi" al blocco di memoria condivisa per poterlo leggere, usando il nome della SHM.
- **Esecuzione Reale:** uguale ai thread
- **Overhead (Pickling):** Quando il processo principale invia i chunk di email ai processi worker, deve "serializzare" (tradurre in byte) i dati per inviarli tramite i canali di comunicazione, e il worker li deve "deserializzare". Queste operazioni hanno un costo non da poco.

I dati raccolti offrono una spiegazione inequivocabile delle differenze prestazionali:

- **Parità di calcolo puro:** La fase di *Esecuzione Reale (Core)*, ovvero il tempo materialmente speso dalla CPU per eseguire l'hashing e l'interrogazione del filtro, risulta pressoché identica tra i due paradigmi (217.78 ms per i thread contro 217.17 ms per i processi). Questo dimostra che l'algoritmo non subisce rallentamenti intrinseci dovuti al modulo `multiprocessing`.
- **Il costo della Memoria Condivisa:** L'istanza dei processi richiede **312.27 ms** unicamente per allocare la memoria a livello di Sistema Operativo (`SharedMemory`) e copiarvi internamente il *bitarray*. I thread, operando nello stesso spazio di indirizzamento dell'interprete padre, saltano completamente questa fase (0 ms).

- **Il blocco della Serializzazione (Pickling):** L'ostacolo più severo per i processi è rappresentato dall'invio degli input ai worker (il partizionamento delle email). Poiché i processi non condividono la RAM, le stringhe inviate ai worker subiscono una fase di serializzazione binaria (*Pickling*), un trasporto Inter-Processo (IPC) e una successiva deserializzazione. Questa operazione "burocratica" ha consumato **901.62 ms**, pari al 62% dell'intero tempo di esecuzione.

In conclusione, la profilazione certifica che per operazioni ad altissima velocità di esecuzione su grandi moli di stringhe in memoria, l'overhead di comunicazione (IPC) e serializzazione richiesto dall'architettura multiprocesso di Python surclassa di svariati ordini di grandezza il tempo del calcolo utile. L'approccio *Free-Threading* (No-GIL), annullando questi costi infrastrutturali, si conferma la scelta architetturale d'elezione.

6. Studio dei FP

Questa parte non è molto rilevante per il corso ma è importante per capire il funzionamento a 360 gradi del Bloom Filter.

Una volta ottenuti dei risultati tra di loro possono esserci pure dei FP (persone a cui è stata bloccata la possibilità di inviare una mail anche se innocue). Ho effettuato una Edit Distance con un set di 5 elementi con tutto il dataset a disposizione per capire se si trattasse di un FP o di un TP. I risultati sono molto interessanti quindi li riporto:

email: erika.zacchia.ufficio@libero.it

Più simile: tonia.zacchia.ufficio@libero.it
(distanza = 4)

Diagnosi: Distante. Probabile collisione casuale pura (FP).

email: loredana.ubaldi.admin@hotmail.com

Più simile: loredana.foconi.admin@hotmail.com
(distanza = 5)

Diagnosi: Distante. Probabile collisione casuale pura (FP).

email: evangelista.terragni.quello@vodafone.it

Più simile: evangelista.errani56@vodafone.it
(distanza = 9)

Diagnosi: Distante. Probabile collisione casuale pura (FP).

email: uberto.sauro.recapito@virgilio.it

Più simile: umberto.emo.recapito@virgilio.it
(distanza = 5)

Diagnosi: Distante. Probabile collisione casuale pura (FP).

email: gloria.squarcione.ufficio@gmail.com

Più simile: lucia.squarcione.ufficio@gmail.com
(distanza = 3)

Diagnosi: Distante. Probabile collisione casuale pura (FP).

Le email targate nella diagnosi (FP) sono una mia suppo-

sizione! potrebbe tranquillamente essere che non è necessario questo tipo di sicurezza e quindi quelle email potrebbero essere TP, tuttavia innegabilmente le email sono effettivamente molto simili tra di loro.

Per classificarle come TP e FP avremmo bisogno di una linea guida di chi vuole effettivamente utilizzare questo strumento.

7. Conclusioni

L'analisi sperimentale condotta evidenzia in modo chiaro come la scelta del paradigma di parallelizzazione in Python influenzi in maniera determinante le prestazioni nella costruzione e interrogazione di un Bloom Filter su larga scala.

In primo luogo, i risultati dimostrano che il parallelismo non rappresenta una soluzione universalmente vantaggiosa: per dataset di piccole dimensioni, l'overhead introdotto (in particolare nei modelli a processi) supera il beneficio computazionale, rendendo l'approccio sequenziale ancora la scelta più efficiente.

All'aumentare della dimensione del dataset, emerge invece un netto cambio: il costo delle operazioni di hashing diventa dominante e consente di ammortizzare l'overhead, rendendo le strategie parallele significativamente più performanti.

Il contributo più rilevante del lavoro riguarda tuttavia l'analisi del parallelismo basato su thread in ambiente No-GIL. I risultati dimostrano che l'eliminazione del Global Interpreter Lock rappresenta un punto di svolta per il calcolo concorrente in Python: i thread riescono a sfruttare efficacemente l'hardware multi-core senza incorrere nei costi di serializzazione e comunicazione tipici del multiprocessing. In particolare, nelle operazioni di interrogazione del Bloom Filter, il throughput raggiunge ottimi miglioramenti fino alla saturazione delle risorse hardware, rendendo questo approccio il più efficiente tra quelli analizzati.

Al contrario, l'uso di processi con memoria condivisa evidenzia limiti strutturali difficilmente superabili. La profilazione ha mostrato come la maggior parte del tempo di esecuzione non sia imputabile al calcolo utile, ma a operazioni di gestione: allocazione della memoria condivisa, serializzazione (pickling) e comunicazione inter-processo.